



第八章:关系数据库设计

数据库系统概念, 6th编辑。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重用条件



第8章:关系型数据库设计

“良好关系设计的特征”

“原子域和第一范式”

“使用功能依赖的分解” 功能依赖理论

“功能依赖” 的算法

“使用多值依赖关系的分解” 更正规的形式

“数据库设计过程”

“时态数据建模”



结合模式?

“假设我们将讲师和部门组合成 *inst_dept* _z (与关系集 *inst_dept* 没有连接)

“结果是信息的可能重复

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



没有重复的组合模式

考虑组合关系

$\mathbf{Z}$ *sec_class(sec_id, building, room_number)*和 $\mathbf{Z}$ *section(course_id, sec_id, semester, year)*成一个关系

$\mathbf{Z}$ *section(course_id, sec_id, semester, year, building, room_number)*

“本例中没有重复



那么更小的schema呢?

“假设我们从`inst_dept`开始。我们如何知道将其拆分(分解)为教员和部门?”

“写一个规则” 如果有一个模式(`dept_name`, `building`, `budget`), 那么`dept_name`将是一个候选键”

“表示为**功能依赖项**:”

$Dept_name \rightarrow \text{建筑, 预算}$

在`inst_dept`中, 由于`dept_name`不是候选键, 因此可能需要重复一个部门的构建和预算。

z 这表明需要分解`inst_dept`

“不是所有的分解都是好的。假设我们将员工(`ID`, 姓名, 街道, 城市, 工资)分解为

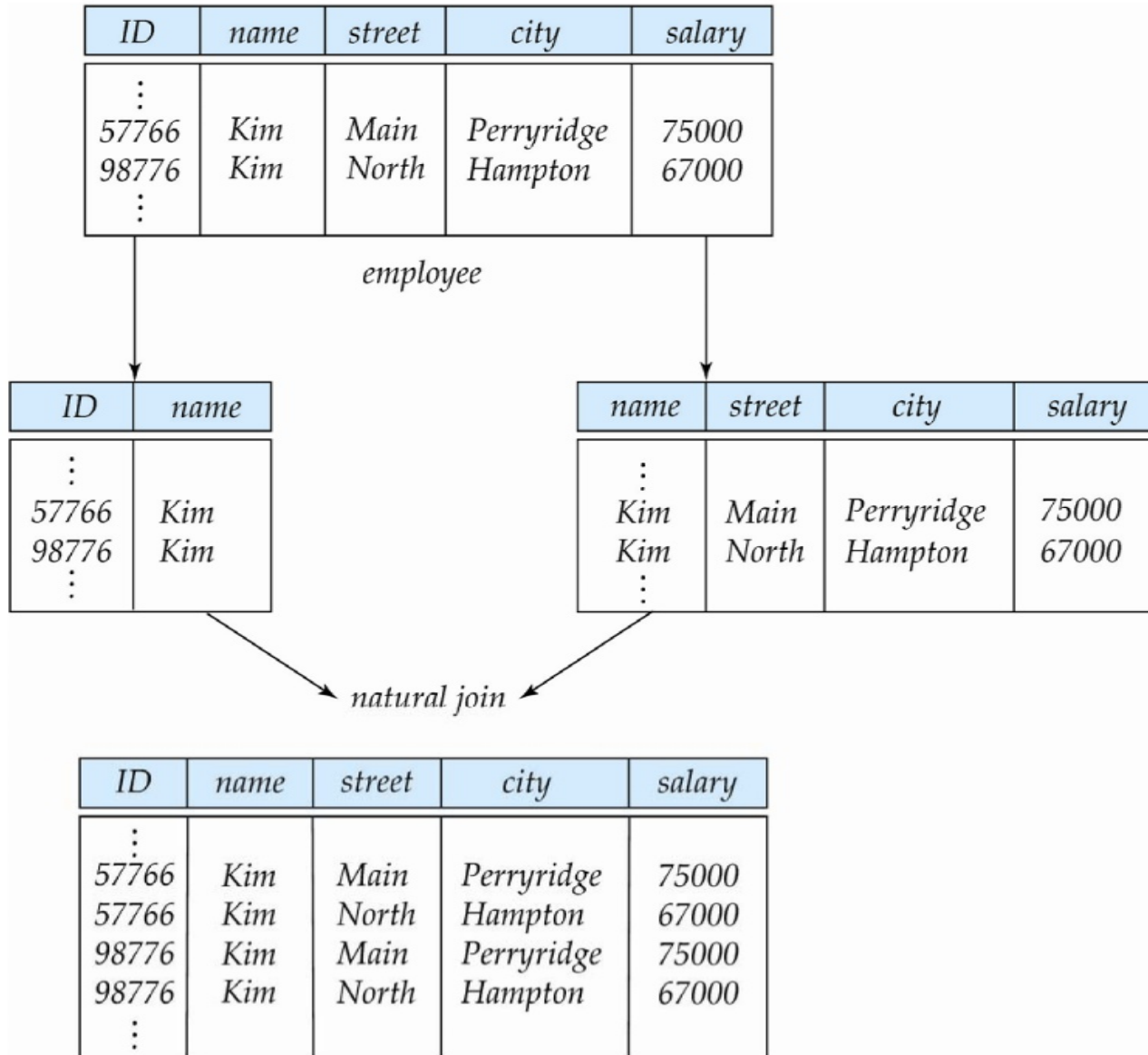
`employee1 (ID, name)`

`Employee2(姓名、街道、城市、工资)`

下一张幻灯片展示了我们是如何丢失信息的——我们无法重建原始的员工关系——所以, 这是一个**有损分解**。



A Lossy Decomposition





Example of Lossless-Join Decomposition

“无损连接分解”

▼ $R = (A, B, C)$ 的分解

$$R_1 = (a, b) \quad R_2 = (b, c)$$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

A	B	C
α	1	A
β	2	B



第一范式

“如果域的元素被认为是不可分割的单位，则域是**原子**的_z非原子域的例子：

名称、复合属性的集合

像CS101这样可以分解成部分的识别号

如果R的所有属性的域都是原子的，那么关系模式R就是**第一范式**

非原子值使存储复杂化，并鼓励数据的冗余(重复)存储

_z示例:存储在每个客户中的一组帐户，以及存储在每个帐户中的一组所有者

我们假设所有的关系都是第一范式(在第22章:基于对象的数据库中重新讨论)



第一范式(经修订)

“原子性实际上是如何使用领域元素的一种属性。

z 示例:字符串通常被认为是不可分割的

假设 给学生的学号是CS0012或EE1127形式的字符串

z 如果提取前两个字符来查找系，则学号的域不是原子的。

z 这样做是一个坏主意:导致在应用程序中编码信息，而不是在数据库中。



目标-为以下内容设计一个理论

确定一个特定关系 R 是否处于“良好”状态。

在关系 R 不处于“良好”形式的情况下，将其分解为一组关系 $\{R_1, R_2, \dots, R_n\}$ 使得

- 每个关系都处于良好状态

- 的分解是一个无损连接分解

我们的理论基于:

- 个功能依赖

- 多值依赖



函数依赖

法律关系集合上的约束。

要求一组属性的值唯一地决定另一组属性的值。

“功能依赖”是键概念的泛化。



功能依赖(续)

“设 R 是一个关系模式

α 、 β 、任任

◆功能依赖性

$$\alpha \rightarrow \beta$$

当且仅当对于任何法律关系 $R(R)$,当 R 的任意两个元组 t_1 和 t_2 在属性 α 上一致时, 它们也在属性 β 上一致。也就是说,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

“示例:考虑 $r(A,B)$ 与以下 r 的实例。

1	4
1	5
3	7

在这个实例中, $A \rightarrow B$ 不成立, 但 $B \rightarrow A$ 成立。



功能依赖(续)

“ K 是关系模式 R 的超键，当且仅当 $K \rightarrow R$ ”

“ K 是 R 的候选键，当且仅当

$\nexists K \rightarrow R$, 且

$\nexists$ 为无 $\alpha \wedge K$, $\alpha \rightarrow R$

功能依赖关系允许我们表达不能用超级键表示的约束。考虑一下模式：

inst_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)。

我们期望这些函数依赖保持不变：

$dept_name \rightarrow 建筑$

和 $ID \nrightarrow building$

但不希望以下内容成立：

$dept_name \rightarrow 工资$



函数依赖的使用

☒ 我们使用功能依赖来:

z 测试关系，看看它们在一组给定的功能依赖下是否合法。

如果一个关系 r 在函数依赖的集合 F 下是合法的，我们说 r 满足 F 。

z 指定了合法关系集合上的约束

我们说，如果 R 上的所有法律关系满足函数依赖集 F ，那么 F 在 R 上成立。

☒ 注意:关系模式的特定实例可能满足功能依赖，即使功能依赖并不适用于所有合法实例。

z 例如，一个特定的讲师实例可能偶然地满足
名称 $\rightarrow$ ID。



功能依赖(续)

如果一个关系的所有实例都满足一个功能依赖，那么它是微不足道的

的例子:

$ID, name \rightarrow ID$

$name \rightarrow name$

一般来说, $\alpha \rightarrow \beta$

当

β

$\geq$

$\geq$

$\geq$

$\geq$

$\geq$

$\geq$

$\geq$

$\geq$

$\geq$

$\geq$



一组功能依赖的闭包

给一个函数依赖的集合 F ，存在某些其他的由 F 逻辑隐含的函数依赖。

例如:如果 $A \rightarrow B$ 和 $B \rightarrow C$ ，那么我们可以推断出 $A \rightarrow C$

“ F 逻辑上隐含的所有功能依赖的集合是 F 的闭包。

我们用 F^+ 表示 F 的闭包。

F^+ 是 F 的超集。



Boyce-Codd Normal Form

关系模式 R 在BCNF中关于函数依赖的集合 F ，如果对于 F^+ 中的所有函数依赖的形式

$$\alpha \rightarrow \beta$$

(1)其中， α 、 β 等，至少符合下列条件之一：

“ $\alpha \rightarrow \beta$ 是微不足道的(即 β 规
模 α)” “ α 是 R 的超关键

示例模式不在BCNF中：

instr_dept (ID, *name*, *salary*, *dept_name*, *building*, *budget*)

因为 $dept_name \rightarrow building, budget$
hold上了*instr_dept*，但 $dept_name$ 不是超级键



将模式分解为BCNF

假设我们有一个模式 R 和一个非平凡的依赖关系 $\alpha \rightarrow \beta$ 导致违反BCNF。

我们将 R 分解为:

- $(\alpha \cup \beta)$
- $(R - (\beta - \alpha))$

在我们的例子中,

$\alpha = dept_name$

$\beta = \text{建筑, 预算}$

而 $inst_dept$ 则被

$(\alpha \cup \beta) = (dept_name, building, budget)$

$(R - (\beta - \alpha)) = (ID, \text{姓名}, \text{工资}, dept_name)$



BCNF和依赖保存

“约束，包括功能依赖，在实践中检查成本很高，除非它们只属于一个关系

如果为了确保所有的功能依赖保持，仅测试分解中每个单独关系上的依赖就足够了，那么该分解是依赖保持的。

由于不可能同时实现BCNF和依赖保持，我们考虑一种较弱的范式，称为第三范式。



第三范式

☒ 关系模式 R 是**第三范式**(3NF), 如果满足以下条件:
在 $F + \alpha \rightarrow \beta$

以下至少有一个成立:

z $\alpha \rightarrow \beta$ 是平凡的(即 $\beta \in \alpha$)

z α 是 R 的超键

z $\beta - \alpha$ 中的每个属性 A 都包含在 r 的候选键中(注:每个属性可能在不同的候选键中)

如果一个关系是BCNF, 那么它就是3NF(因为在BCNF中, 必须满足上述前两个条件之一)。

第三个条件是最小限度地放松BCNF, 以确保依赖性保留(稍后将看到原因)。



归一化目标

- ☒ 设 R 是一个函数依赖集 F 的关系方案。判断一个关系方案 R 是否处于“良好”的形式。
- ☒ 在关系方案 R 不处于“良好”形式的情况下，将其分解为一组关系方案 $\{R_1, R_2, \dots, R_n\}$ 使得 γ 每个关系方案都处于良好形式
- γ 的分解是一个无损连接分解
- 最好，分解应该是保持依赖关系的。



BCNF有多好?

BCNF中有一些数据库模式似乎没有得到充分的规范化

考虑一个关系

inst_info (*ID*, *child_name*, *phone*)

z, 一个指导员可能有多个电话, 可以有多个孩子

ID	child_name	phone
99999	David	512-555-1234
99999	David	512-555-4321
99999	William	512-555-1234
99999	Willian	512-555-4321

inst_info



BCNF有多好?(续)。

不存在非平凡的函数依赖关系，因此该关系属于BCNF

“插入异常-即。，如果我们将电话981-992-3443添加到99999，我们需要添加两个元组

(99999, David, 981-992-3443)

(99999, William, 981-992-3443)



How good is BCNF? (Cont.)

因此，最好将`inst_info`分解为:

inst_child
inst_phone

ID	child_name
99999	David
99999	David
99999	William
99999	Willian

ID	phone
99999	512-555-1234
99999	512-555-4321
99999	512-555-1234
99999	512-555-4321

这表明需要更高的范式，比如第四范式(4NF)，我们将在后面看到。



功能相关的理论

我们现在考虑形式理论，它告诉我们哪些功能依赖是由一组给定的功能依赖在逻辑上隐含的。

然后我们开发算法来生成BCNF和3NF的无损分解

然后，我们开发算法来测试分解是否保持依赖关系



一组功能依赖的闭包

给一个函数依赖集合 F ，有一些其他的函数依赖在逻辑上由 F 隐含。z For e.g.:如果 $a \rightarrow B$ 和 $B \rightarrow C$ ，那么我们可以推断 $a \rightarrow C$

“ F 逻辑上隐含的**所有**功能依赖的集合是 F 的**闭包**。”

我们用 F^+ 表示 F 的闭包。



一组功能依赖的闭包

“我们可以通过反复应用**阿姆斯壮公理**，找到F的闭包

F^+ :

z 若 β 蕴含 α ，则 $\alpha \rightarrow \beta$ (反身性)

z if $\alpha \rightarrow \beta$ ，则 $\gamma\alpha \rightarrow \gamma\beta$ (增强性)

z if $\alpha \rightarrow \beta$ ，和 $\beta \rightarrow \gamma$ ，则 $\alpha \rightarrow \gamma$ (传递性)

“这些规则是

z **音** (只生成实际持有的功能依赖)，以及

z **完备** (生成所有持有的功能依赖)。



例子

“ $r = (a, b, c, g, h, i)$

$F = \{ \rightarrow B$

$\rightarrow C$

$CG \rightarrow h$

$CG \rightarrow i$

$B \rightarrow H \}$

F^+ 的一些成员

$z \rightarrow H$

通过传递性从 $A \rightarrow B$ 和 $B \rightarrow H$

$z AG \rightarrow I$

通过 $A \rightarrow C$ 加 G , 得到 $AG \rightarrow CG$

然后是 $CG \rightarrow I$ 的及物性

$z CG \rightarrow HI$

通过增大 $CG \rightarrow I$ 来推导 $CG \rightarrow CGI$, 增大 $CG \rightarrow H$ 来
推导 $CGI \rightarrow HI$,

然后是传递性



计算过程 F^+

☒ 要计算一组函数依赖 F 的闭包:

$F^+ = F$

重复

对于 F^+ 中的每个函数依赖项 f

对 f 应用自反性和增广规则

将结果函数依赖项添加到 F^+

对于 F^+ 中的每一对函数依赖 f_1 and f_2

如果 f_1 和 f_2 可以使用传递性组合, 则将结果函数依赖项添
加到 F^+ 中

直到 F^+ 不再改变

注意:我们将在后面看到这个任务的另一个过程



功能依赖的闭包(续)

附加规则:

z 如果 $\alpha \rightarrow \beta$ 保持和 $\alpha \rightarrow \gamma$ 保持, 则 $\alpha \rightarrow \beta\gamma$ 保持(并集) z 如果 $\alpha \rightarrow \beta\gamma$ 保持, 则 $\alpha \rightarrow \beta$ 保持和 $\alpha \rightarrow \gamma$ 保持(分解)

z 如果 $\alpha \rightarrow \beta$ hold 和 $\gamma\beta \rightarrow \delta$ hold, 则 $\alpha\gamma \rightarrow \delta$ hold

(pseudotransitivity)

以上规则可以从阿姆斯特朗公理中推断出来。



属性集的闭包

给定一组属性 α ，定义 α 在 F 下的**闭包**(用 α^+ 表示)为 F 下由 α 在功能上决定的属性集

⊠ Algorithm to compute α^+ , the closure of α under F

```
result :=  $\alpha$ ;  
while (changes to result) do  
  for each  $\beta \rightarrow \gamma$  in  $F$  do  
    begin  
      if  $\beta \subseteq \text{result}$  then result := result  $\cup \gamma$   
    end
```



属性集闭包的例子

* $r = (a, b, c, g, h, i)$

“ $f = \{a \rightarrow b$
 $\rightarrow C$
 $CG \rightarrow h$
 $CG \rightarrow i$
 $B \rightarrow H\}$

$(AG)^+$

1. $result = AG$
2. $result = ABCG$ ($A \rightarrow C$ 和 $A \rightarrow B$)
3. $result = ABCGH$ ($CG \rightarrow H$ 、 CG)
4. $result = ABCGHI$ ($CG \rightarrow I$ 和 CG) $\leq AG$ 是否为候选关键?

1. AG Is吗?

1. $AG \rightarrow R? = Is (AG)^+ R$

2. Is AG 的任何子集都是超密钥? 1. $A \rightarrow R$ 吗? $= Is (A)^+ R$

2. $G \rightarrow R$ 吗? $= Is (G)^+ R$



属性闭包的使用

属性闭包算法有几种用法:



对superkey的测试:

z 为了测试 α 是否为超键, 我们计算 α^+ , 并检查 α^+ 是否包含 R 的所有属性。



测试函数依赖性



z 要检查一个函数依赖 $\alpha \rightarrow \beta$ 是否成立(或者, 换句话说, 是否在 F^+ 中), 只需检查 $\beta \subseteq \alpha^+$ 。

z 也就是说, 我们通过使用属性闭包来计算 α^+ , 然后检查它是否包含 β 。

z 是一个简单廉价的测试, 非常有用

F 的计算闭包

z 对于每一个 $\gamma \in R$, 我们求出闭包 γ^+ , 对于每一个 $S \in \gamma^+$, 我们输出一个函数依赖 $\gamma \rightarrow S$ 。



规范的封面

☒ 功能依赖的集合可能有冗余的依赖，可以从其他依赖中推断出来

例如: $A \rightarrow C$ 在 $\{A \rightarrow B, B \rightarrow C, A \not\rightarrow C\}$ 中是冗余的

函数依赖的部分可能是冗余的

示例: 在 RHS 上: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ 可以简化为

$\{a \rightarrow b, b \rightarrow c, a \rightarrow d\}$

例: on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ 可简化为

$\{a \rightarrow b, b \rightarrow c, a \rightarrow d\}$

直观地说，F 的一个规范覆盖是等价于 F 的功能依赖的“最小”集合，没有冗余依赖或冗余部分的依赖



附加属性

考虑功能依赖的集合 F 和 F 中的功能依赖 $\alpha \rightarrow \beta$ 。

如果 $A \in \alpha$, 则 A 属性与 α 无关

而 F 在逻辑上意味着 $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ 。

如果 $A \in \beta$, 则 A 属性与 β 无关

和函数依赖的集合

$(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ 逻辑上蕴涵着 F 。

注:在上述每种情况下, 相反方向的暗示都是微不足道的, 因为“更强”的函数依赖总是意味着更弱的函数依赖

例:给定 $F = \{A \rightarrow C, AB \rightarrow C\}$

B 在 $AB \rightarrow C$ 中是无关的, 因为 $\{A \rightarrow C, AB \rightarrow C\}$ 在逻辑上意味着 $A \rightarrow C$ (即从 $AB \rightarrow C$ 中去掉 B 的结果)。

例:给定 $F = \{A \rightarrow C, AB \rightarrow CD\}$

C 在 $AB \rightarrow CD$ 中是无关的, 因为即使删除 C 也可以推断出 $AB \rightarrow C$



测试一个属性是否为Extraneous

考虑功能依赖的集合 F 和 F 中的功能依赖 $\alpha \rightarrow \beta$ 。

为了测试属性 $A \in \alpha$ 是否在 α 中是无关的

1. 使用 F 中的依赖项计算 $(\{\alpha\} - A)^+$
2. 检查 $(\{\alpha\} - A)^+$ 中是否含有 β ;为了检验属性 $A \in \beta$ 在 β 中是否无关
 1. 仅使用中的依赖项计算 α^+
$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - a)\},$$
 2. 检查 α^+ 是否含有 A ;如果是这样, A 在 β 中是无关的



规范的封面

F 的**规范覆盖**是一组依赖项 F_c , 使得 F 逻辑上暗示了 F_c 中的所有依赖项, 并且

F_c 逻辑上暗示 F 中的所有依赖项, 和

F_c 中没有功能依赖项包含外部属性, 并且 F_c 中功能依赖项的左侧每个都是唯一的。

为了计算 F 的规范覆盖:

重复

使用联合规则将 F $\alpha_1 \rightarrow \beta_1$ 和 $\alpha_1 \rightarrow \beta_2$ 中的任何依赖替

换为 $\alpha_1 \rightarrow \beta_1 \beta_2$
在 α 或 β 中找到一个具有无关属性的功能依赖

项 $\alpha \rightarrow \beta$

/*注意:使用 F_c 测试无关属性, 而不是 F^* /
如果发现无关属性, 从 $\alpha \rightarrow \beta$ 中删除它

直到 F 不变

注:在删除了一些无关的属性后, 联合规则可能会变得适用, 因此必须重新应用



计算规范覆盖

* $r = (a, b, c)$

$F = \{a \rightarrow BC$

$B \rightarrow C$

$\rightarrow B$

$AB \rightarrow c\}$

将 $A \rightarrow BC$ 和 $A \rightarrow B$ 合并为 $A \rightarrow BC$

z 集现在是 $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$ {A 在 $AB \rightarrow C$ 中是无关的}

检查从 $AB \rightarrow C$ 删除 A 的结果是否隐含在其他依赖项中

Yes: 其实 $B \rightarrow C$ 已经存在了!

z 集现在是 $\{A \rightarrow BC, B \rightarrow C\}$

☒ C 与 $A \rightarrow BC$ 无关

z 检查 $A \rightarrow C$ 是否在逻辑上由 $A \rightarrow B$ 和其他依赖关系隐含是: 在 $A \rightarrow B$ 和 $B \rightarrow C$ 上使用及物性。

-可以在更复杂的情况下使用 A 的属性闭包

☒ 规范的封面是: $A \rightarrow B$

$B \rightarrow C$



Lossless-join分解

对于 $R = (R_1, R_2)$ 的情况，我们要求对于模式 R 上的所有可能关系 R

$$R = \Pi_{R_1}(R) \bowtie \Pi_{R_2}(R)$$

如果以下依赖项中至少有一个在 F^+ 中，则将 R 分解为 R_1 和 R_2 是无损连接：

$$\text{z } R_1 \cap R_2 \rightarrow R_1$$

$$\text{z } R_1 \cap R_2 \rightarrow R_2$$

上面的函数依赖性是无损连接分解的充分条件；只有当所有约束都是功能依赖时，这些依赖项才是必要条件



例子

* $r = (a, b, c)$

$F = \{a \rightarrow b, b \rightarrow c\}$

z 可以用两种不同的方式分解 “ $R1 = (A, B), R2 = (B, C)$ ”

z 无损连接分解:

$$R1 \cap R2 = \{B\} \text{ and } B \rightarrow BC$$

z 依赖保持

“ $r_1 = (a, b), r_2 = (a, c)$ ”

z 无损连接分解:

$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$

z 不依赖保持

(不计算 R 不能检查 $B \rightarrow C_1$

⋈ R_2)



依赖保护

设 F_i 为仅包含 R_i 中的属性的依赖项 F^+ 的集合。

一个分解是**依赖保持的**，如果 $(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$

如果不是，那么检查更新是否违反函数依赖可能需要计算连接，这是昂贵的。



依赖性保存测试

为了检查依赖项 $\alpha \rightarrow \beta$ 是否在 R 分解为 $R_1, R_2, \dots, R_n$ 中保留，我们应用以下测试(对 F 进行属性闭包)

z结果 = α

While(更改为 $result$) do

对于分解中的每个 R_i

$$T = (\text{结果} \cap R_i)^+ \cap R_i$$

$$Result = Result \cup t$$

z如果 $result$ 包含 β 中的所有属性，则函数依赖 $\alpha \rightarrow \beta$ 是保存。

我们对 F 中的所有依赖项应用测试，以检查分解是否保持依赖

这个过程需要多项式时间，而不是计算 F^+ 和 $(F_1 \cup F_2 \cup \dots \cup F_n)^+$ 所需的指数时间



例子

* $r = (a, b, c)$

$F = \{a \rightarrow b$

$B \rightarrow C\}$

键 = $\{A\}$

“ R ” 不在BCNF中

在BCNF中分解 $R1 = (A, B), R2 = (B, C)$ $\bowtie$ $R1$

和 $R2$

$\bowtie$ 无损连接分解 (lossless-join decomposition)

$\bowtie$ 依赖保持



BCNF测试

检查非平凡依赖项 $\alpha \rightarrow \beta$ 是否会导致违反BCNF

1. 计算 α^+ (α 的属性闭包), 和
2. 验证它包含 R 的所有属性, 即它是 R 的超键。

简化测试:为了检查一个关系模式 R 是否在BCNF中, 只检查给定集合 F 中的依赖项是否违反BCNF, 而不是检查 F^+ 中的所有依赖项。

如果 F 中的依赖项都不违反BCNF, 那么 F^+ 中的依赖项也不会违反BCNF。

然而, 在测试 R 分解中的关系时, 仅使用 F 的简化测试是不正确的

考虑 $R = (A, B, C, D, E)$, 其中 $F = \{A \rightarrow B, BC \rightarrow D\}$ 将 R 分解为 $R_1 = (A, B)$ 和 $R_2 = (A, C, D, E)$

F 中的两个依赖项都不包含来自 (A, C, D, E) 的属性, 因此我们可能被误导认为 R_2 满足BCNF。

事实上, F^+ 中的依赖项 $AC \rightarrow D$ 显示 R_2 不在BCNF中。



BCNF分解测试

- ☒ 为了检验 R 分解中的关系 R_i 是否在BCNF中，
 - z 根据 F 对 R_i 的**限制**(即 F^+ 中只包含 R_i)属性的所有fd)对BCNF进行 R_i 测试
 - z 或使用保持在 R 上的原始依赖集 F ，但使用以下测试：
 - 对于每一组属性 $\alpha \subseteq R_i$ ，检查 α^+ (α 的属性闭包)要么不包含 $R_i - \alpha$ 的属性，要么包含 R_i 的所有属性。
- 如果 F 中的某些 $\alpha \rightarrow \beta$ 违反了条件，则依赖关系
$$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$$
可以显示hold住 R_i ，而 R_i 违反了BCNF。
- 我们使用上面的依赖关系来分解 R_i



BCNF分解算法

```
result := {R };
done := false;
compute F+;
while (not done) do
  if (there is a schema Ri in result that is not in BCNF)
    then begin
      let  $\alpha \rightarrow \beta$  be a nontrivial functional dependency that
        holds on Ri such that  $\alpha \rightarrow R_i$  is not in F+,
        and  $\alpha \cap \beta = \emptyset$ ;
      result := (result - Ri)  $\cup$  (Ri -  $\beta$ )  $\cup$  ( $\alpha, \beta$ );
    end
  else done := true;
```

注意:每个 R_i 都在BCNF中, 并且分解是无损连接的。



BCNF分解的例子

* $r = (a, b, c)$

$F = \{a \rightarrow b$
 $B \rightarrow C\}$

键 = $\{A\}$

“ R 不在BCNF中($B \rightarrow C$ 但 B 不是超级键)” 分解

$R_1 = (B, C)$

$R_2 = (A, B)$



BCNF分解的例子

“class (course_id, title, dept_name, 学分, sec_id, 学期, 学年, building, room_number, capacity, time_slot_id)

“功能依赖关系:

$\text{course_id} \rightarrow \text{title, dept_name, 学分}$

$\text{building, room_number} \rightarrow \text{capacity}$

$\text{course_id, sec_id, semester, year} \rightarrow \text{building, room_number, time_slot_id}$

“一个候选键{course_id, sec_id, semester, year}。

BCNF分解:

$\text{course_id} \rightarrow \text{title, dept_name, 学分}$ 持有

但course_id不是超级键。

我们将class替换为:

课程(course_id, title, dept_name, 学分)

class-1 (course_id, sec_id, semester, year, building, room_number, capacity, time_slot_id)



BCNF分解(续)

☒ 课程在BCNF

z我们是怎么知道的?

{*building, room_number* → *capacity* 持有 1 类 z 但 {*building, room_number*} 不是 1 类的超键。z 我们将类-1 替换为:

classroom (*building, room_number, capacity*)

section (*course_id, sec_id, semester, year, building, room_number, time_slot_id*)

*教室和分部在BCNF。



BCNF and Dependency Preservation

得到一个保持依赖关系的BCNF分解并不总是可能的

“ $r = (j, k, l)$ ”

$F = \{JK \rightarrow l$

$L \rightarrow K\}$

两个候选键 = JK 和 JL

“ R ” 不在BCNF中

对 R 的任何分解都不能保存

$JK \rightarrow L$

这意味着测试 $JK \rightarrow L$ 需要一个连接



第三范式:动机

- ☒ 有些情况下
 - z BCNF不是保持依赖关系的, 并且
- 解决方案:定义一个较弱的范式, 称为第三范式(3NF)
- ☒ z 允许一些冗余(由此产生的问题;我们将在后面看到例子)
 - z 但是函数依赖可以在不计算连接的情况下检查单个关系。
 - z 总是有一个无损连接, 保持依赖关系的3NF分解。



3NF的例子

☒ *dept_advisor*关系:

dept_advisor (*s_ID*, *i_ID*, *dept_name*)

$F = \{s_ID, dept_name \rightarrow i_ID, i_ID \rightarrow dept_name\}$ 两个候选

键:*s_ID*, *dept_name*, *i_ID*, *s_ID* R 为3NF

$s_ID, dept_name \rightarrow i_ID$ s_ID

-*dept_name*为超级键

$i_ID \rightarrow dept_name$

-*dept_name*包含在候选键中



3NF冗余

“在这个模式中有一些冗余” 3NF中冗余的问题示例

$R = (J, K, L) F = \{JK \rightarrow L, L \rightarrow K\}$

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

重复的信息(例如, 关系 l_1, k_1) $(i_ID, dept_name)$

需要使用空值(例如, 表示关系 l_2, k_2 , 其中 J 没有对应的值)。

$(i_ID, dept_name)$, 如果没有单独的关系映射指导员到部门



3NF测试

优化:只需要检查 F 中的 fd , 不需要检查 F^+ 中的所有 fd 。

如果 α 是超键, 则使用属性闭包检查每个依赖项 $\alpha \rightarrow \beta$ 。

如果 α 不是超级键, 我们要验证 β 中的每个属性是否包含在 R 的候选键中

ω 这个测试的开销相当大, 因为它涉及到寻找候选键

3NF的 ω 测试显示为NP-hard

ω 有趣的是, 分解成第三范式(稍后描述)可以在多项式时间内完成



3NF分解算法

设 F_c 为 F 的一个标准封面; $I:= 0$;

对于 F_c 中的每个功能依赖 $\alpha \rightarrow \beta$ 做如果没有模式 R_j ,

$1 \leq j \leq i$ 包含 $\alpha\beta$

然后开始

$I:= I + 1$;

$R_i:= \alpha\beta$

结束

如果没有模式 R_j , 则 $1 \leq j \leq i$ 包含 R 的候选键

然后开始

$I:= I + 1$;

$R_i:= R$ 的任意候选键;结束

/*可选, 删除冗余关系*/

重复

如果任何模式 R_j 包含在另一个模式 R_k 中, 则/*删除

R_j */

$R_j = r$;;

我张=;

return ($R_1, R_2, \dots$ 国际扶轮)



3NF分解算法(续)

- ☒ 上述算法保证:
 - z 每个关系模式 R_i 都在3NF中
 - z 分解是依赖保持和无损连接z 正确性证明在本演示的最后(点击[这里](#))
-



3NF分解举例

“关系模式:

$Cust_banker_branch = (\underline{customer_id, employee_id}, branch_name, type)$

这个关系模式的功能依赖项是:

1. $Customer_id, employee_id \rightarrow branch_name, type$
2. $employee_id \rightarrow branch_name$
3. $Customer_id, branch_name \rightarrow employee_id$

我们首先计算一个canonical cover

z $branch_name$ 在1st依赖项的r.h.s.中是无关的 z 没有其他属性是无关的, 所以我们得到 $F_C = customer_id, employee_id \rightarrow type$

$Employee_id \rightarrow branch_name$ $customer_id,$
 $branch_name \rightarrow Employee_id$



3NF分解示例(Cont.)

“**for** 循环生成以下 3NF 模式 :(*customer_id*,
employee_id, *type*)

(*employee_id* *branch_name*)

(*customer_id*, *branch_name*, *employee_id*)

z 观察到(*customer_id*, *employee_id*, *type*)包含一个原模式的候选键，因此不需要再添加关系模式

“在for循环结束时，检测并删除模式，例如(*employee_id*, *branch_name*),
这些模式是其他模式的子集

z 的结果将不依赖于fd被考虑的顺序，得到的简化3NF模式是:

(*customer_id*, *employee_id*, *type*)

(*customer_id*, *branch_name*, *employee_id*)



BCNF与3NF的比较

将一个关系分解为一组符合3NF的关系总是可能的，以便：

z分解是无损的

依赖性保留的

总是有可能将一个关系分解为BCNF中的一组关系，以便：

z分解是无损的

可能不可能保持依赖关系。



设计目标

关系数据库设计的目标是:

- z BCNF。
- z 无损联接。
- z 依赖保存。

如果不能做到这一点，我们就接受其中之一

- z 缺乏依赖保存
- z 使用3NF造成的冗余

有趣的是，除了超级键之外，SQL没有提供指定功能依赖的直接方法。

可以使用断言指定fd，但是测试起来成本很高，(而且目前没有任何广泛使用的数据库支持!)

即使我们有一个保持依赖关系的分解，使用SQL我们也不能有效地测试一个左边不是键的功能依赖。



多值依赖

“假设我们记录孩子的名字，以及教师的电话号码：

z *inst_child*(*ID*, *child_name*)

z *inst_phone*(*ID*, *phone_number*)

如果我们将这些模式组合起来得到

z *inst_info*(*ID*, *child_name*, *phone_number*)

z 样例数据：

(99999, David, 512-555-1234)

(99999, David, 512-555-4321)

(99999, William, 512-555-1234)

(99999, William, 512-555-4321)



这个关系在BCNF中

z 为什么？



多值依赖关系(mvd)

☒ 设 R 为关系图式，设 α 、 β 任任。多值依赖性

$$\alpha \twoheadrightarrow \beta$$

如果在任何法律关系 $R(R)$ 中,对于 R 中的元组 t_1 和 t_2 的所有对,使得 $t_1[\alpha] = t_2[\alpha]$, 则 R 中存在元组 t_3 和 t_4 , 使得:

$$t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$$

$$t_3[\beta] = t_1[\beta] \\ t_3[R-\beta] = t_2[R-\beta] \quad t_4$$

$$t_4[\beta] = t_2[\beta] \\ t_4[R-\beta] = t_1[R-\beta]$$



MVD (Cont.)

“ $\alpha \twoheadrightarrow \beta$ ”的表格表示。

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$



例子

设 R 是一个关系模式，其属性集被划分为3个非空子集。

Y, z, w

我们说 $Y \twoheadrightarrow Z$ (Y 多重决定 Z)

当且仅当对于所有可能的关系 r (r)

$\langle y1, z1, w1 \rangle \in r$ 和 $\langle y1, z2, w2 \rangle \in r$

然后

$\langle y1, z1, w2 \rangle \in r$ 和 $\langle y1, z2, w1 \rangle \in r$

注意，由于 Z 和 W 的行为是相同的，因此如果 $Y \twoheadrightarrow W$ ，则 $Y \twoheadrightarrow Z$



例子(续)。

在我们的例子中:

$$ID \twoheadrightarrow child_name \text{ and } ID \twoheadrightarrow phone_number$$

上面的形式化定义应该形式化这样一个概念: 给定一个特定的Y值(ID), 它已经与它关联了一组Z值(*child_name*)和一组W值(*phone_number*), 并且这两个集合在某种意义上是相互独立的。

☒注意:

如果 $Y \rightarrow z$ 则 $Y \twoheadrightarrow z$

确实我们有(在上面的符号中) $z_1 = z_2$
结论如下。



多值依赖关系的使用

我们以两种方式使用多值依赖:

1. 测试关系, 以**确定**它们在给定的一组功能和多值依赖项下是否合法
2. 规定对法律关系集合的**约束**。因此, 我们将只关注满足一组给定的功能和多值依赖的关系。

如果关系 r 不能满足给定的多值依赖, 我们可以通过向 r 中添加元组来构造一个满足多值依赖的关系 r' 。



mvd的理论

☒ 从多值依赖的定义中，我们可以推导出以下规则：

如果 $\alpha \rightarrow \beta$ ，则 $\alpha \rightarrow \rightarrow \beta$

也就是说，每一个泛函依赖也是一个多值依赖， D 的闭包 D^+ 是 D 逻辑隐含的所有泛函依赖和多值依赖的集合。



z 我们可以用函数依赖和多值依赖的形式化定义，从 D 中计算 D^+ 。

z 我们可以用这样的推理来管理非常简单的多值依赖关系，这在实践中似乎是最常见的

z 对于复杂的依赖，最好使用推理规则系统来推理依赖集(参见附录C)。





第四范式

如果对 D^+ 中形式为 $\alpha \twoheadrightarrow \beta$ 的所有多值依赖项, 且 α 和 β $\geq$ 一个, 则关系模式 R 对泛函多值依赖项集 $D \geq 4NF$, 其中:

$\alpha \twoheadrightarrow \beta$ 是平凡的(即 $\beta \subseteq \alpha$ 或 $\alpha \cup \beta = R$)

α 是模式 R 的超键

如果一个关系是在 $4NF$ 中, 它就是在 $BCNF$ 中



多值依赖的限制



D对 R_i 的限制是集合 D_i 由

D^+ 中只包含 R_i 属性的所有功能依赖形式的所有多值依赖

$$\alpha \twoheadrightarrow \beta \cap R_i$$

哪里 $\alpha \subseteq R_i$ 和 $\alpha \twoheadrightarrow \beta$ 在 D^+



4NF分解算法

```
result := {R};  
done := false;  
compute  $D^+$ ;  
Let  $D_i$  denote the restriction of  $D^+$  to  $R_i$   
while (not done)  
  if (there is a schema  $R_i$  in result that is not in 4NF) then  
    begin  
      let  $\alpha \twoheadrightarrow \beta$  be a nontrivial multivalued dependency that holds  
      on  $R_i$  such that  $\alpha \rightarrow R_i$  is not in  $D_i$ , and  $\alpha \cap \beta = \phi$ ;  
      result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );  
    end  
  else done := true;
```

Note: each R_i is in 4NF, and decomposition is lossless-join





Example

“ $r = (a, b, c, g, h, i)$

$F = \{a \twoheadrightarrow b \quad b \twoheadrightarrow$

$h \quad c \twoheadrightarrow g \quad g \twoheadrightarrow h\}$

由于 $A \twoheadrightarrow B$ 和 A 不是 R 的超键, 因此 R 不在4NF中

☒ 分解

a) $R_1 = (a, B)$ (R_1 为4NF)

b) $R_2 = (A, C, G, H, I)$ (R_2 不在4NF内, 分解为 R_3 和 R_4)) $R_3 = (C, G, H)$ (R_3 在4NF内)

d) $R_4 = (A, C, G, I)$ (R_4 不在4NF中, 分解为 R_5 和 R_6)

$\text{z } A \twoheadrightarrow B \text{ 和 } B \twoheadrightarrow HI \hat{=} A \twoheadrightarrow HI, \text{ (MVD传递率) 和}$

z

因此 $A \twoheadrightarrow I$ (MVD限制为 R_4)

e) $R_5 = (A, I)$

$G)(R_5$ 为4NF)

f) $R_6 = (A, C,$

$(R_6$ 为4NF)



进一步的Normal Forms

联接依赖概括了多值依赖

导致项目-连接范式(PJNF)(也称为第五范式)

一类更一般的约束，导致一种称为域键范式的范式。

“这些广义约束的问题:很难推理，并且不存在一套健全完整的推理规则集。

因此很少使用



数据库总体设计流程

- ☒ 我们假设模式 R 是给定的
 - z 可以在将E-R图转换为一组表时生成。
 - z R 可以是一个包含所有感兴趣的属性的单一关系(称为**通用关系**)。
 - z 归一化将 R 分解成更小的关系。
 - z R 可能是一些特殊关系设计的结果, 然后我们测试/转换为标准形式。



ER模型和归一化

当仔细设计E-R图，正确识别所有实体时，从E-R图生成的表不应需要进一步规范化。

然而，在真实的(不完美的)设计中，实体的非关键属性与实体的其他属性之间可能存在功能依赖关系

z 示例: 具有 *department_name* 和 *building* 属性的
员工实体，
和一个功能依赖
department_name → 建筑

z 好的设计会使 *department* 成为一个实体
来自关系集的非关键属性的功能依赖是可能的，但很少见——大多数关系是二元的



性能的反规格化

可能需要使用非规范化的模式来提高性能

“例如，将`prereqs`与`course_id`一起显示，并且`title`当然需要与`prereq`连接

“替代方案1:使用包含`course`属性的非规范化关系以及具有上述所有属性的
`preeq`

z更快的查找

z额外的空间和额外的执行时间用于更新

z程序员的额外编码工作和额外代码中的错误可能性»替代方案2:使用定义的
物化视图

当然`prereq`

z与上述相同的优点和缺点，除了没有额外的编码工作，程序员和避免可
能的错误





其他设计问题

- ❑ 数据库设计的一些方面没有被规范化抓住，数据库设计不好的例子，要避免：

代替 *earnings* (*company_id*, *year*, *amount*), use

z *earnings_2004*, *earnings_2005*, *earnings_2006* 等，都在模式 (*company_id*, *earnings*) 上。

以上都在BCNF中，但是使得跨年查询变得困难，每年都需要新表

z *company_year* (*company_id*, *earnings_2004*, *earnings_2005*, *earnings_2006*)

同样在BCNF中，但也使得跨年查询变得困难，并且每年需要新的属性。

是一个交叉表的例子，其中一个属性的值变成了列名
用于电子表格和数据分析工具



建模时态数据

“时态数据具有数据有效的关联时间间隔。

“快照”是数据在特定时间点的值

“通过向 z 属性添加有效时间来扩展ER模型的几个建议，例如，教师在 z 实体不同时间点的地址，例如，学生实体存在的时间持续时间

z 关系，例如，讲师作为指导老师与学生相关联的时间。

但没有公认的标准

增加一个时间成分会导致功能依赖，比如 $ID \rightarrow \text{street, city}$

不hold，因为地址随时间变化

如果函数依赖项 $X \twoheadrightarrow Y$ 保留所有合法实例 $R(R)$ 的所有快照，则暂时函数依赖项 $X \twoheadrightarrow Y$ 保留模式 R 。



建模时态数据(Cont.)

在实践中，数据库设计人员可能会为关系添加开始和结束时间属性

z 例如, $course(course_id, course_title)$ 被 $course(course_id, course_title, start, end)$ 所取代

约束: 没有两个元组可以有重叠的有效时间-很难有效地执行

“外键引用可能是对当前版本的数据，也可能是对某个时间点的数据

z 例如，学生成绩单应参考学习该课程时的课程信息



章节结束

数据库系统概念，6thEd。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重新使用的条件



3NF分解算法的正确性证明

数据库系统概念，6th编辑。

©Silberschatz, Korth和Sudarshan参见www.db-book.com了解
重新使用的条件



3NF分解算法的正确性

- ☒ 3NF分解算法是保持依赖关系的(因为 F_c)中每个FD都有关系)
分解是无损的
- ☒
 - z 一个候选键(C)在分解中的一个关系 R_i 中 zF_c 下的候选键的闭包必须包含 R 中的所有属性。
 - z 按照属性闭包算法的步骤来表示有
在 R_i 中每个元组的联接结果中只有一个元组



3NF分解算法的正确性(续)

声明:若上述算法生成的分解中存在关系 R_i , 则 R_i 满足3NF。

▼设 R_i 由依赖关系 $\alpha \rightarrow \beta$ 生成

▼设 $\gamma \rightarrow B$ 为 R_i 上的任意非平凡函数依赖(我们只需要考虑其右边是单个属性的fd。)

“现在, B 可以在 β 或 α 中任选一个, 但不能同时在两个属性中。
分别考虑每一种情况。



3NF分解的正确性(续)

“情况1:如果 β 中有 B :

如果 γ 是超键, 则满足3NF的第二个条件

否则 α 必须包含一些不在 γ 中的属性

由于 $\gamma \rightarrow B$ 在 F^+ 中, 它一定可以从 F_c 中导出, 通过对 γ 使用属性闭包。

z 属性闭包没有使用过 $\alpha \rightarrow \beta$ 。如果使用过, α 一定包含在 γ 的属性闭包中, 这是不可能的, 因为我们假设 γ 不是超键。

z 现在, 利用 $\alpha \rightarrow (\beta - \{B\})$ 和 $\gamma \rightarrow B$, 我们可以推出 $\alpha \rightarrow B$
(因为 $\gamma \supseteq \alpha\beta$, 且 $B \in \gamma \supseteq \gamma \rightarrow B$ 是非平凡的)

z 则, 在 $\alpha \rightarrow \beta$ 的右侧, B 是无关的;这是不可能的, 因为 $\alpha \rightarrow \beta$ 在 F_c 中。

因此, 如果 B 在 β 中, 则 γ 必须是超键, 且必须满足3NF的第二个条件。



3NF分解的正确性(续)

- ☒ 情况2: B 在 α 。
 - 由于 α 是一个候选键，因此3NF定义中的第三种选择通常是满足的。
 - 事实上，我们不能证明 γ 是一个超键。
 - 这正好说明了为什么在3NF的定义中会出现第三种选择。

Q.E.D.



Figure 8.02

<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000



Figure 8.03

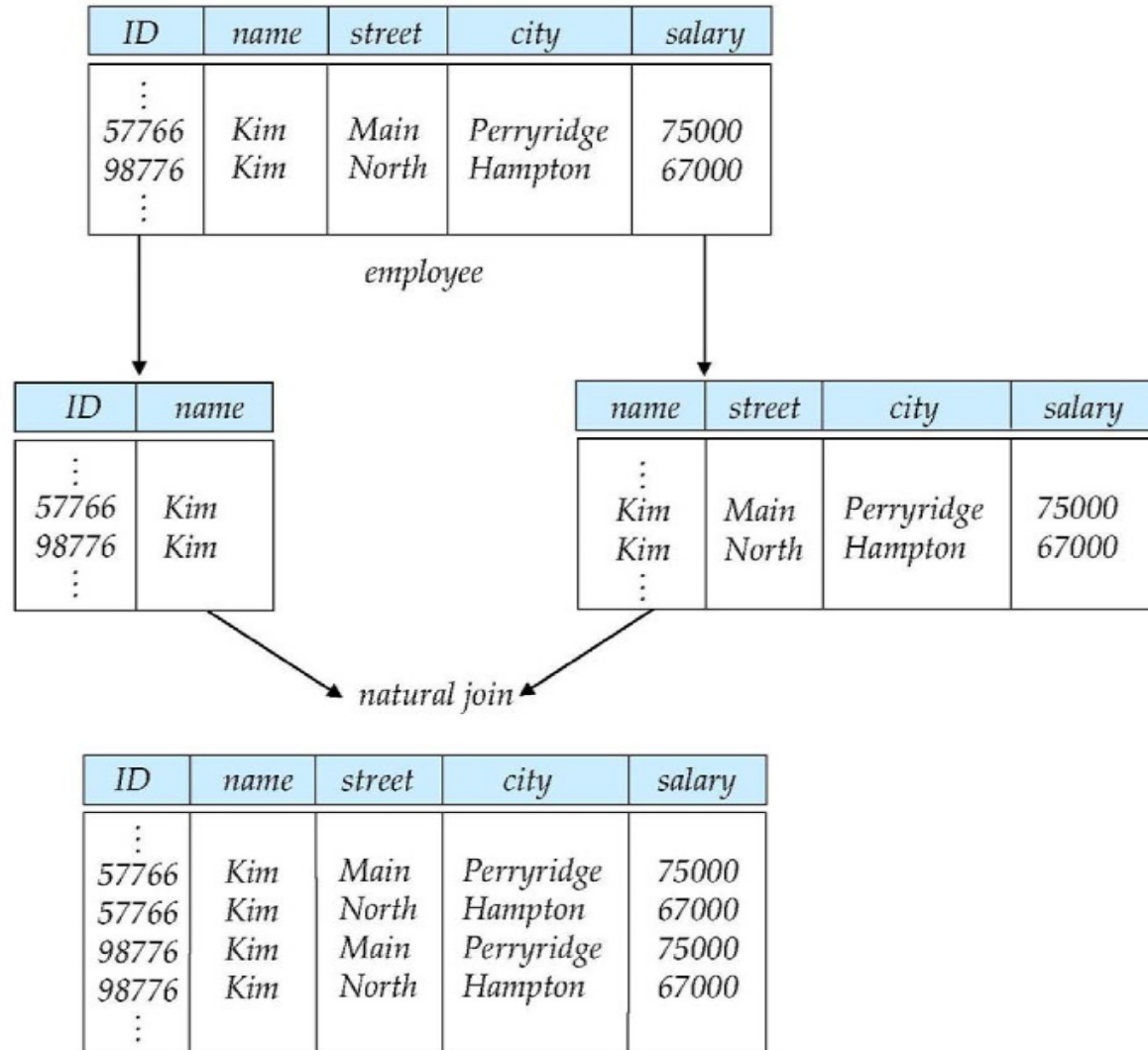




Figure 8.04

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_1	d_2
a_2	b_2	c_2	d_2
a_2	b_3	c_2	d_3
a_3	b_3	c_2	d_4



图8.05

<i>building</i>	<i>room_number</i>	<i>capacity</i>
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50



Figure 8.06

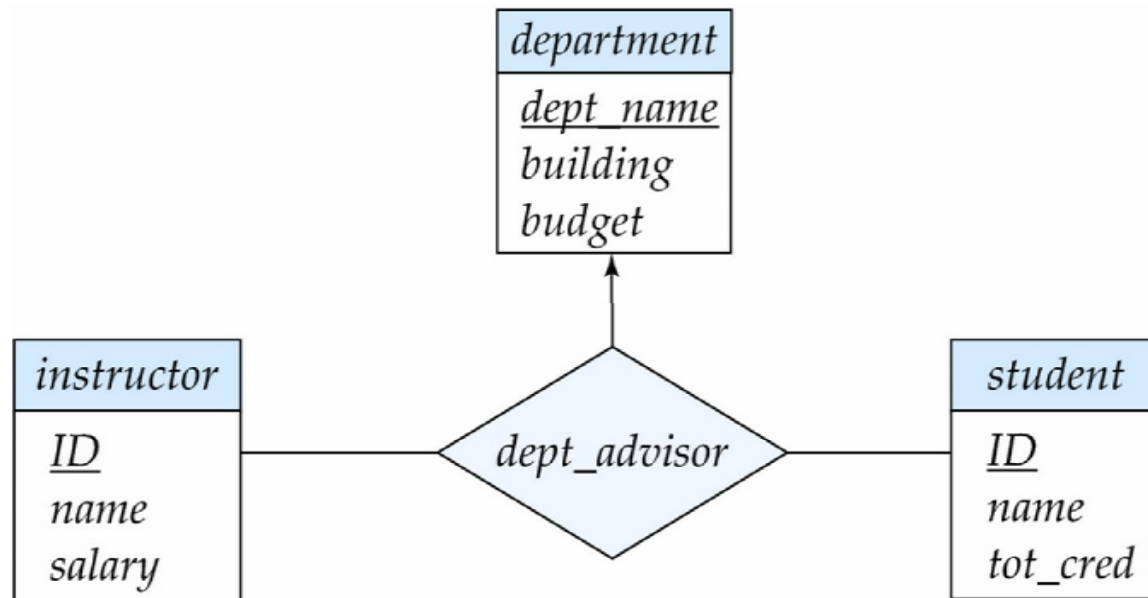




图8.14

<i>dept_name</i>	<i>ID</i>	<i>street</i>	<i>city</i>
Physics	22222	North	Rye
Physics	22222	Main	Manchester
Finance	12121	Lake	Horseneck



图8.15

<i>dept_name</i>	<i>ID</i>	<i>street</i>	<i>city</i>
Physics	22222	North	Rye
Math	22222	Main	Manchester



图8.17

A	B	C
a_1	b_1	c_1
a_1	b_1	c_2
a_2	b_1	c_1
a_2	b_1	c_3